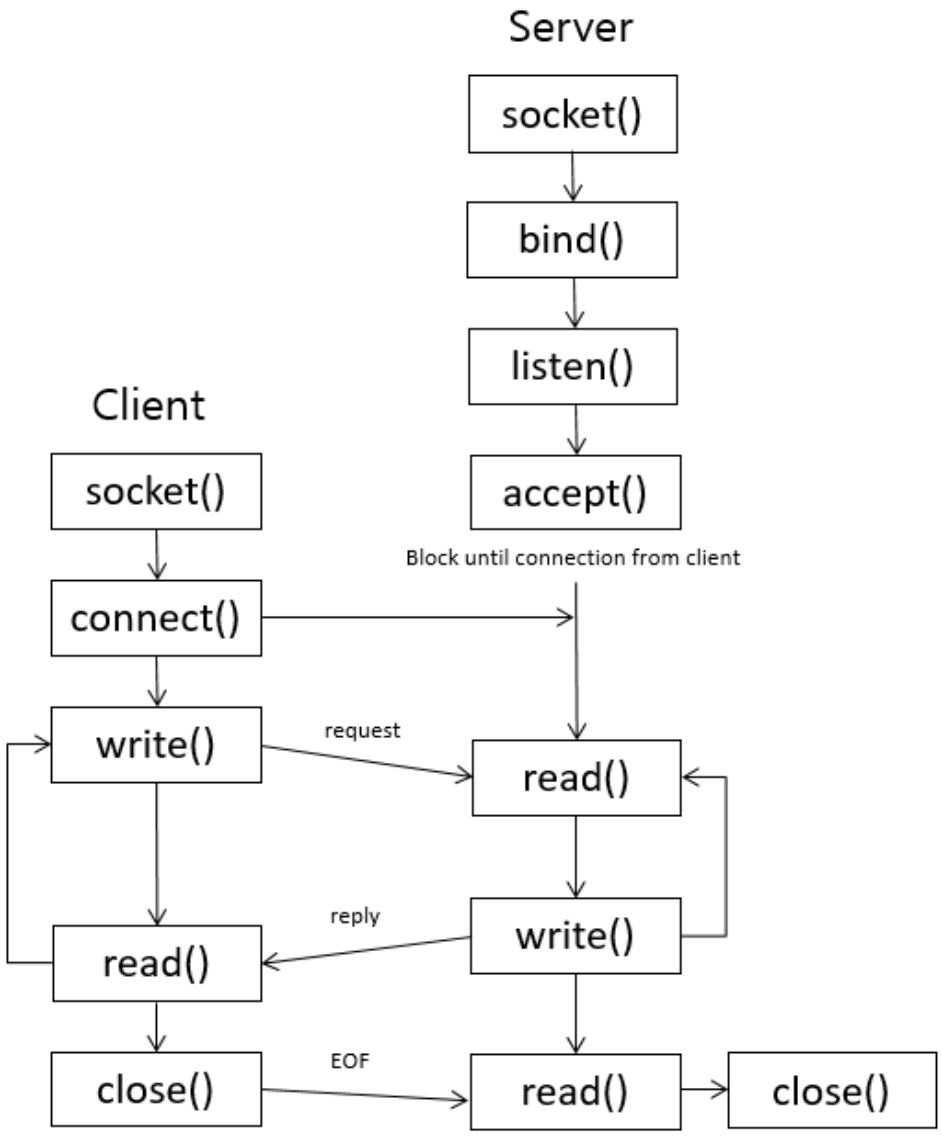
BHARATIYA VIDYA BHAVAN'S SARDAR
PATEL INSTITUTE OF TECHNOLOGY
(Empowered Autonomous Institute Affiliated to Mumbai University)
Department Of Computer Engineering

Name	Krish Sodhi, Marvin Sukumar, Riya Suryawanshi, Anushka Suvarna
UID	2023300233, 2023300238, 2023300240, 2023300241
Subject	Distributed Computing
Experiment No.	2
Project title	Travel Itinerary Planner System
Problem Statement	Implementation of Client Server Communication using Socket Programming, RPC and RMI.
Objectives	1. To use Socket Programming so that clients (like mobile or desktop users) can send requests to the Travel Itinerary server and get responses back, allowing two-way communication over the network. 2. To use RPC (Remote Procedure Call) so that the client can call functions like authenticate to verify user credentials 3. To allow programs running on different computers to call and execute functions on remote machines, we are using RMI.
Theory	1. **Socket Programming:** In our Travel Itinerary System, socket programming helps clients (like mobile or desktop users) send requests to servers (such as Authentication, Restaurants, Events, Transport, and Attractions) and receive responses. For example, when a user searches for "restaurants in Mumbai," the client program opens a socket connection to the Restaurant Server, sends the request, and receives the list of restaurants. This communication happens in real time, just like chatting in a messaging app, but instead of messages, we are sending travel data. Important socket functions: ● socket(): Creates a socket, an endpoint for network communication. ● bind(ip, port): Binds the socket to a server IP address and port. ● listen(): Makes the server wait for client requests. ● accept(): Accepts a connection from a client. ● read() and write() (or send() and recv()): Exchange data between client and server. ● close(): Closes the connection after communication is finished.

On the client side:

- socket(): Creates a socket.
- connect(server_address, server_port): Connects to the server.
- send() and recv(): Send data to server and receive response.
- close(): Ends the connection.



2. **RPC (Remote Procedure Call)**

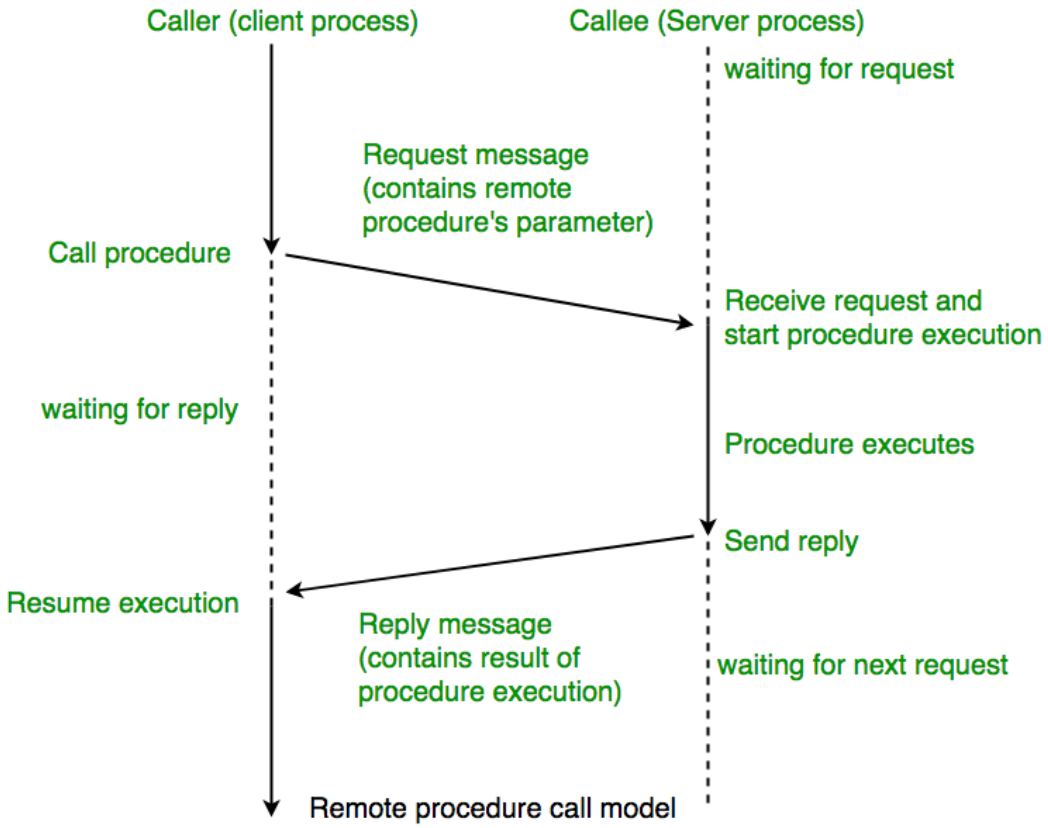
In our project, RPC allows the client to call functions on a remote server as if they were local.

For example, when the user wants to check "events in Delhi," instead of writing the whole event-fetching code inside the client, the client simply calls getEvents("Delhi"). Behind the scenes, this request goes to the Event Server, the function executes there, and the result is returned to the client.

This makes our system simpler and modular because different services (restaurants, events, transport) can run independently, but still work together seamlessly.

Main elements of RPC:

- Client: Starts the call.
- Client Stub: Packs the parameters and sends them over the network.
- Server Stub: Unpacks the request and prepares it for execution.
- Server: Executes the requested procedure and sends the result back.



3. **RMI (Remote Method Invocation)**

In our Travel Itinerary System, we used Java RMI to allow different services (Auth, Restaurant, Event, Transport, Attraction servers) to run on separate machines, while the client (Coordinator) can call their methods directly.

For example, the client can call:

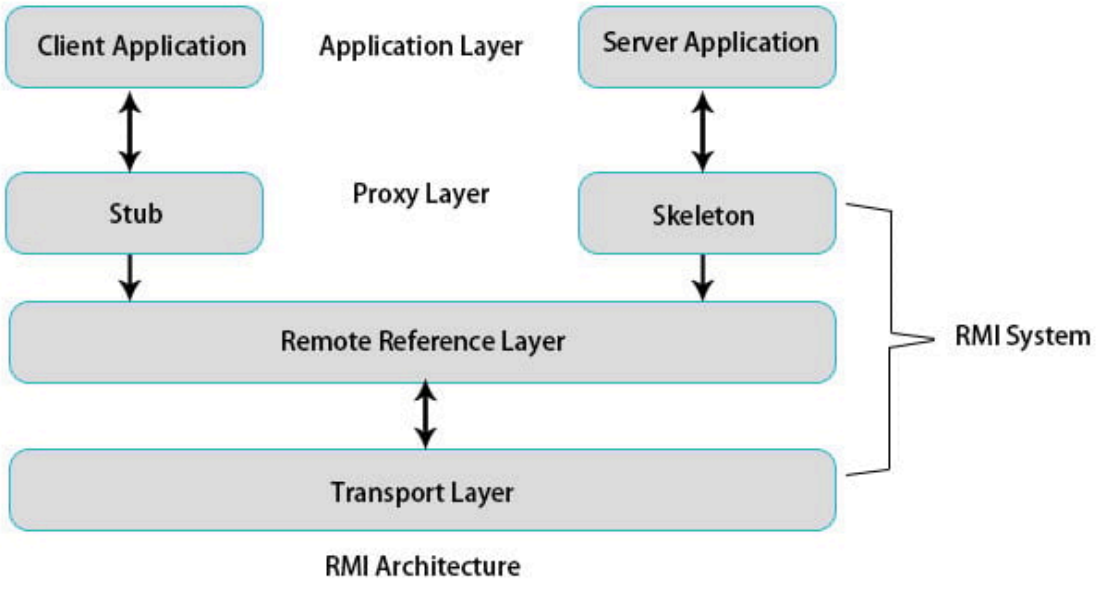
1. auth.authenticate("alice", "password"): to login.
2. restaurant.getRestaurants("Mumbai"): to fetch restaurants.
3. event.getEvents("Delhi"): to fetch events.
4. transport.getTransport("Mumbai", "Pune"): to get transport routes.
5. attraction.getAttractions("Mumbai"): to get attractions.

This looks like normal method calls in Java, but actually happens remotely over the network.

Main components of RMI:

- Stub (Client side): Acts as a gateway. Sends the client's request to the server.
- Skeleton (Server side): Receives the request, calls the actual method, and returns the result.
- Remote Object: The actual object running on the server that executes the method.

This makes our Travel Itinerary System scalable (each service can run independently) and modular (easy to add new services in the future).



RMI Architecture

Socket Programming:

a. Server Side:

```
import socket
import json
from datetime import datetime

# Example "user database"
users = {
    "admin": "1234",
    "alice": "password",
    "bob": "qwerty"
}

def authenticate(username, password):
    return users.get(username) == password

def start_auth_server():
    host = "127.0.0.1"
    port = 5000

    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.bind((host, port))
    server_socket.listen(1)
    print(f"[AuthServer] Listening on {host}:{port}")

    conn, addr = server_socket.accept()
    print(f"[AuthServer] Connection from {addr}")

    while True:
        data = conn.recv(1024).decode()
        if not data:
            break

        try:
            request = json.loads(data)  # Expect {"username": "...", "password": "..."}
            username = request.get("username")
            password = request.get("password")


            print(f"[{datetime.now()}] [AuthServer] Login attempt by '{username}'")

            if authenticate(username, password):
                response = {"status": "AUTH_SUCCESS"}
                print(f"[AuthServer] User '{username}' authenticated successfully ")
            else:
                response = {"status": "AUTH_FAILED"}
                print(f"[AuthServer] Authentication failed for '{username}' ")

        except Exception as e:
            response = {"status": "ERROR", "message": str(e)}
```

```
            print(f"[AuthServer] Error: {e}")

        conn.send(json.dumps(response).encode())

    conn.close()

if __name__ == "__main__":
    start_auth_server()
```

Client Side:

```
import socket
import json

def coordinator():
    host = "127.0.0.1"
    port = 5000

    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client_socket.connect((host, port))
    print("[Coordinator] Connected to Auth Server")

    while True:
        username = input("Enter username: ")

        password = input("Enter password: ")

        credentials = {"username": username, "password": password}
        client_socket.send(json.dumps(credentials).encode())

        response = json.loads(client_socket.recv(1024).decode())
        print("[Coordinator] Response:", response)

        if response.get("status") == "AUTH_SUCCESS":
            print("Logged in Successfully! Start making your itinerary...\n")
            break
        else:
            print("Login Failed, check your credentials and try again.\n")

    client_socket.close()
    print("[Coordinator] Disconnected.")

if __name__ == "__main__":
    coordinator()
```

OUTPUT:

```
(base) PS C:\Users\ASUS\Documents\Study\Distributed_Computing\Socket> python auth_server
[AuthServer] Listening on 127.0.0.1:5000
[AuthServer] Connection from ('127.0.0.1', 49788)
[2025-08-26 17:27:01.188699] [AuthServer] Login attempt by 'admin'
[AuthServer] Authentication failed for 'admin'
[2025-08-26 17:27:06.229873] [AuthServer] Login attempt by 'admin'
[AuthServer] User 'admin' authenticated successfully
```

```
(base) PS C:\Users\ASUS\Documents\Study\Distributed_Computing\Socket> python coordinator.p
[Coordinator] Connected to Auth Server
Enter username: admin
Enter password: 123
[Coordinator] Response: {'status': 'AUTH_FAILED'}
Login Failed, check your credentials and try again.

Enter username: admin
Enter password: 1234
[Coordinator] Response: {'status': 'AUTH_SUCCESS'}
Logged in Successfully! Start making your itinerary...
```

RPC Code and Output:

Server Side:

```
from xmlrpc.server import SimpleXMLRPCServer

users = {
    "admin": "1234",
    "alice": "password",
    "bob": "qwerty"
}

def authenticate(username, password):
    if users.get(username) == password:
        return "AUTH_SUCCESS"
    else:
        return "AUTH_FAILED"

def start_auth_server():
    server = SimpleXMLRPCServer(("localhost", 7001))
    print("Hi !!!\nAuth server Starting\n[AuthServer] XML-RPC Server listening on port 9000...")
    server.register_function(authenticate, "authenticate")
    server.serve_forever()

if __name__ == "__main__":
    start_auth_server()
```

Client Side:

```
import xmlrpc.client

def coordinator(username,password):
    proxy = xmlrpc.client.ServerProxy("http://localhost:7001/")
    print("[Coordinator] Connected to Auth Server via XML-RPC")

    response = proxy.authenticate(username, password)
    print("[Coordinator] Response:", response)
    if response=="AUTH_SUCCESS":
        print("Logged in Successfully, \nStart Making your first itinerary!!!")
        return True
    else:
```

```
        print("Login Failed, check your credentials and try again")
        return False


if __name__ == "__main__":
    print("Welcome to Travel Itinerary Planner!!!")

    resp = False
    while resp!=True:
        username = input("\nEnter your username:\n")
        password = input("\nEnter Password:\n")
        resp = coordinator(username,password)
```

Output:

```
(base) PS C:\Users\ASUS\Documents\Study\Distributed_Computing\RPC> python auth_server.py
Hi !!!
Auth server Starting
[AuthServer] XML-RPC Server listening on port 9000...
127.0.0.1 - - [25/Aug/2025 19:16:35] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [25/Aug/2025 19:18:30] "POST / HTTP/1.1" 200 -
```

```
(base) PS C:\Users\ASUS\Documents\Study\Distributed_Computing\RPC> python coordinator
Welcome to Travel Itinerary Planner!!!

Enter your username:
admin

Enter Password:
123
[Coordinator] Connected to Auth Server via XML-RPC
[Coordinator] Response: AUTH_FAILED
Login Failed, check your credentials and try again

Enter your username:
admin

Enter Password:
1234
[Coordinator] Connected to Auth Server via XML-RPC
[Coordinator] Response: AUTH_SUCCESS
Logged in Successfully,
Start Making your first itinerary!!!
```

RMI code and Output:

Client Side:

```
import java.rmi.Naming;
import java.util.Scanner;

public class CoordinatorClient {
   public static void main(String[] args) {
      try {

         AuthInterface auth = (AuthInterface)
Naming.lookup("rmi://localhost:1099/AuthService");
         System.out.println("[Coordinator] Connected to Auth Server via RMI");

         Scanner scanner = new Scanner(System.in);

         while (true) {
            System.out.print("Enter username (or 'exit' to quit): ");
            String username = scanner.nextLine();

            if (username.equalsIgnoreCase("exit")) {
               System.out.println("[Coordinator] Exiting client...");
               break;
            }

            System.out.print("Enter password: ");
            String password = scanner.nextLine();

            String response = auth.authenticate(username, password);
            System.out.println("[Coordinator] Response: " + response);

            if (response.equals("AUTH_SUCCESS")) {
               System.out.println(" Logged in Successfully!\nStart Making your first
itinerary!!!");
               break;
            } else {
               System.out.println(" Login Failed, check your credentials and try again.\n");
            }
         }

         scanner.close();
```

```
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Server Side:

```
import java.rmi.Naming;
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;
import java.util.HashMap;

public class AuthServer extends UnicastRemoteObject implements AuthInterface {
    private HashMap<String, String> users;

    protected AuthServer() throws RemoteException {
        super();
        users = new HashMap<>();
        users.put("admin", "1234");
        users.put("alice", "password");
        users.put("bob", "qwerty");
    }

    public String authenticate(String username, String password) throws
RemoteException {
        if (users.containsKey(username) && users.get(username).equals(password)) {
            System.out.println("[AuthServer] Login SUCCESS for user: " + username);
            return "AUTH_SUCCESS";
        } else {
            System.out.println("[AuthServer] Login FAILED for user: " + username);
            return "AUTH_FAILED";
        }
    }

    public static void main(String[] args) {
        try {
            AuthServer server = new AuthServer();
            Naming.rebind("rmi://localhost:1099/AuthService", server);
            System.out.println("Server Starting... \n[AuthServer] RMI Server is running...");
```

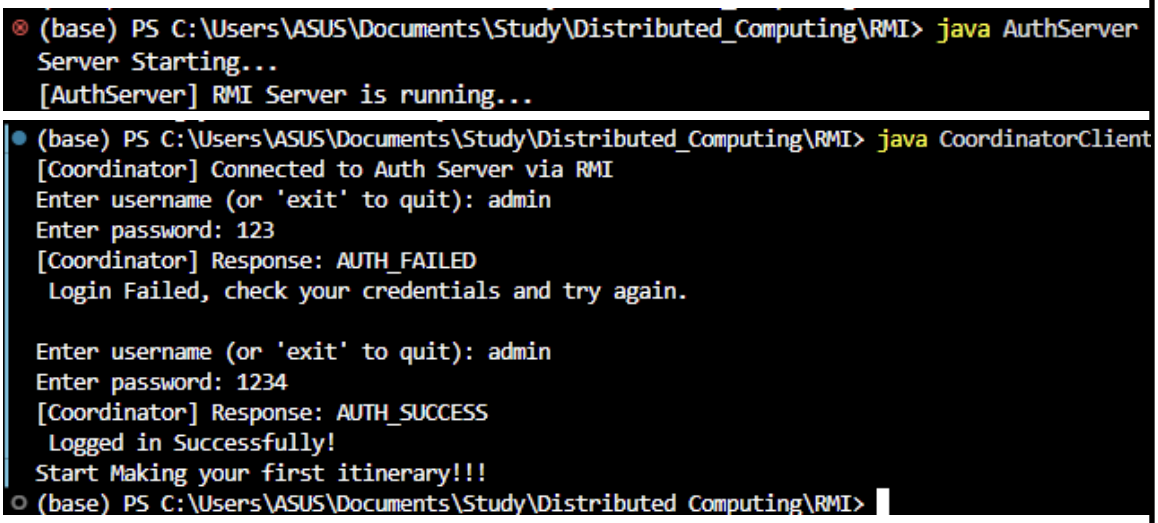
```
        } catch(Exception e) {
            e.printStackTrace();
        }
    }
}
```

Remote Interface:

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface AuthInterface extends Remote {
    String authenticate(String username, String password) throws RemoteException;
}
```

Output:



Conclusion:

By completing this project, we successfully implemented client-server communication for our Travel Itinerary Planner System using Socket Programming, RPC, and RMI. These technologies helped establish reliable connections between clients and the server, enabled smooth exchange user credentials and verifying user. The system thus demonstrates how distributed communication techniques can be applied to build a practical, interactive, and user-friendly travel planning application.